

Homework4

February 18, 2019

```
In [1]: import numpy as np
import scipy.stats
import pandas as pd
import sklearn as sk
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import calibration_curve
from sklearn.metrics import confusion_matrix
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
import matplotlib
import matplotlib.pyplot as plt
from matplotlib import cm
import pygam
from pygam import LinearGAM
from pygam import LogisticGAM
```

```
In [2]: import warnings
warnings.filterwarnings(action='ignore')
```

```
In [3]: matplotlib.rcParams['figure.figsize'] = (9,6)
```

1 Problem 1

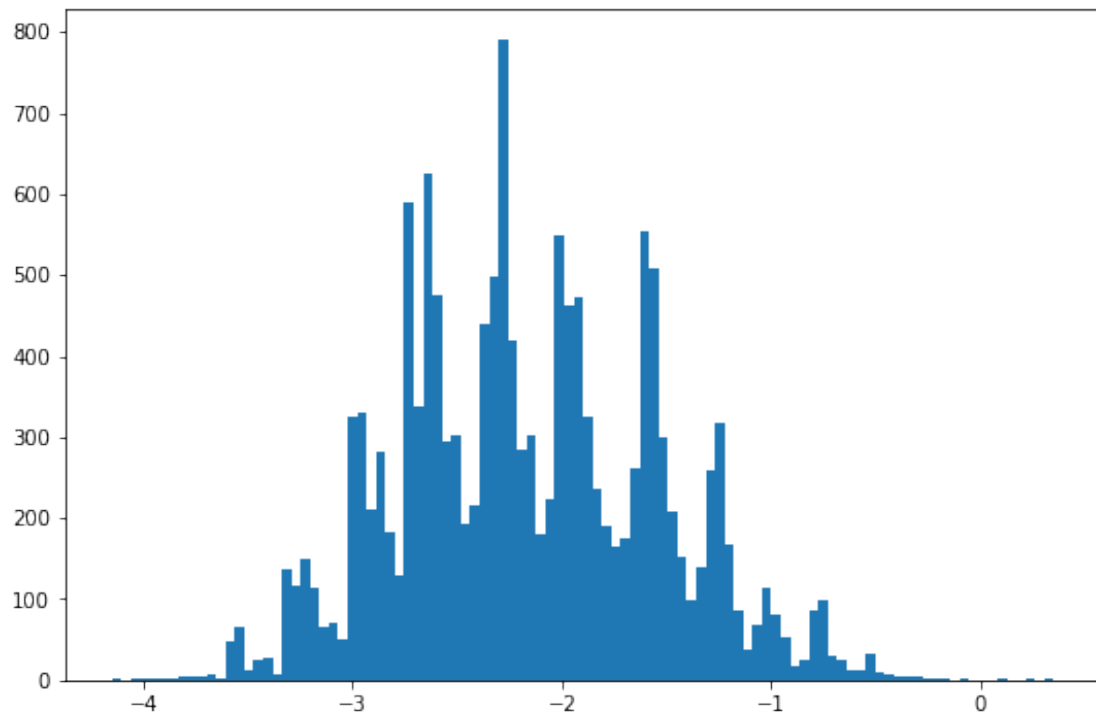
(a)

```
In [5]: market_data = pd.read_csv('./marketing.csv')
Y = np.where(market_data['y'] == 'yes', 1, 0)
X = market_data.drop(['y'], axis = 1)
X = pd.get_dummies(X)
xtrain, xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.33, random_state=1)

logistic_model = LogisticRegression(C = 100000)
log_model_fit = logistic_model.fit(xtrain, ytrain)
y_pred = log_model_fit.predict(xtest)
y_prob = log_model_fit.predict_proba(xtest)[:,:1]
```

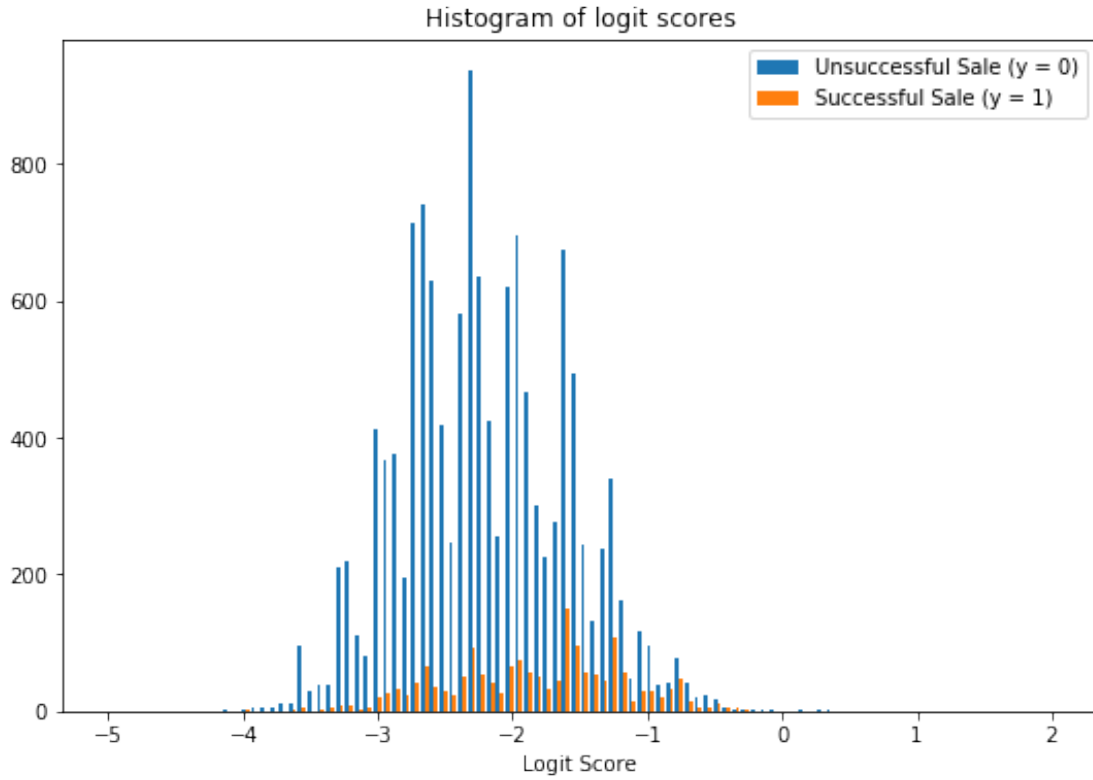
```
logit_score = np.log(y_prob/(1-y_prob))
plt.hist(logit_score,bins = 100)
```

```
Out[5]: (array([ 1.,  0.,  1.,  2.,  1.,  3.,  3.,  4.,  4.,  5.,  8.,
  3., 48., 65., 13., 26., 27.,  7., 137., 116., 150., 114.,
 65., 70., 50., 324., 331., 210., 283., 182., 129., 590., 339.,
625., 476., 295., 303., 194., 215., 439., 497., 790., 420., 285.,
303., 179., 224., 549., 462., 473., 324., 237., 189., 166., 176.,
261., 555., 508., 299., 208., 151., 99., 139., 259., 318., 167.,
 86., 37., 69., 115., 82., 53., 18., 26., 87., 98., 30.,
 26., 12., 12., 33., 10.,  8.,  4.,  4.,  4.,  3.,  1.,
  2.,  0.,  1.,  0.,  0.,  0.,  1.,  0.,  0.,  1.,  0.,
  1.]),
array([-4.14894335, -4.10391559, -4.05888783, -4.01386007, -3.96883231,
 -3.92380455, -3.87877679, -3.83374903, -3.78872127, -3.74369351,
 -3.69866575, -3.65363799, -3.60861023, -3.56358247, -3.51855471,
 -3.47352695, -3.4284992 , -3.38347144, -3.33844368, -3.29341592,
 -3.24838816, -3.2033604 , -3.15833264, -3.11330488, -3.06827712,
 -3.02324936, -2.9782216 , -2.93319384, -2.88816608, -2.84313832,
 -2.79811056, -2.7530828 , -2.70805504, -2.66302728, -2.61799952,
 -2.57297177, -2.52794401, -2.48291625, -2.43788849, -2.39286073,
 -2.34783297, -2.30280521, -2.25777745, -2.21274969, -2.16772193,
 -2.12269417, -2.07766641, -2.03263865, -1.98761089, -1.94258313,
 -1.89755537, -1.85252761, -1.80749985, -1.76247209, -1.71744434,
 -1.67241658, -1.62738882, -1.58236106, -1.5373333 , -1.49230554,
 -1.44727778, -1.40225002, -1.35722226, -1.3121945 , -1.26716674,
 -1.22213898, -1.17711122, -1.13208346, -1.0870557 , -1.04202794,
 -0.99700018, -0.95197242, -0.90694466, -0.86191691, -0.81688915,
 -0.77186139, -0.72683363, -0.68180587, -0.63677811, -0.59175035,
 -0.54672259, -0.50169483, -0.45666707, -0.41163931, -0.36661155,
 -0.32158379, -0.27655603, -0.23152827, -0.18650051, -0.14147275,
 -0.09644499, -0.05141723, -0.00638948,  0.03863828,  0.08366604,
  0.1286938 ,  0.17372156,  0.21874932,  0.26377708,  0.30880484,
  0.3538326 ]),
<a list of 100 Patch objects>)
```



(b)

```
In [7]: y_0 = logit_score[ytest == 0]
        y_1 = logit_score[ytest == 1]
        bins = np.linspace(-5,0,10)
        plt.hist([y_0, y_1], 100, label=['Unsuccessful Sale (y = 0)', 'Successful Sale (y = 1)'])
        plt.xlabel('Logit Score')
        plt.title(r'Histogram of logit scores')
        plt.legend()
        plt.show()
```



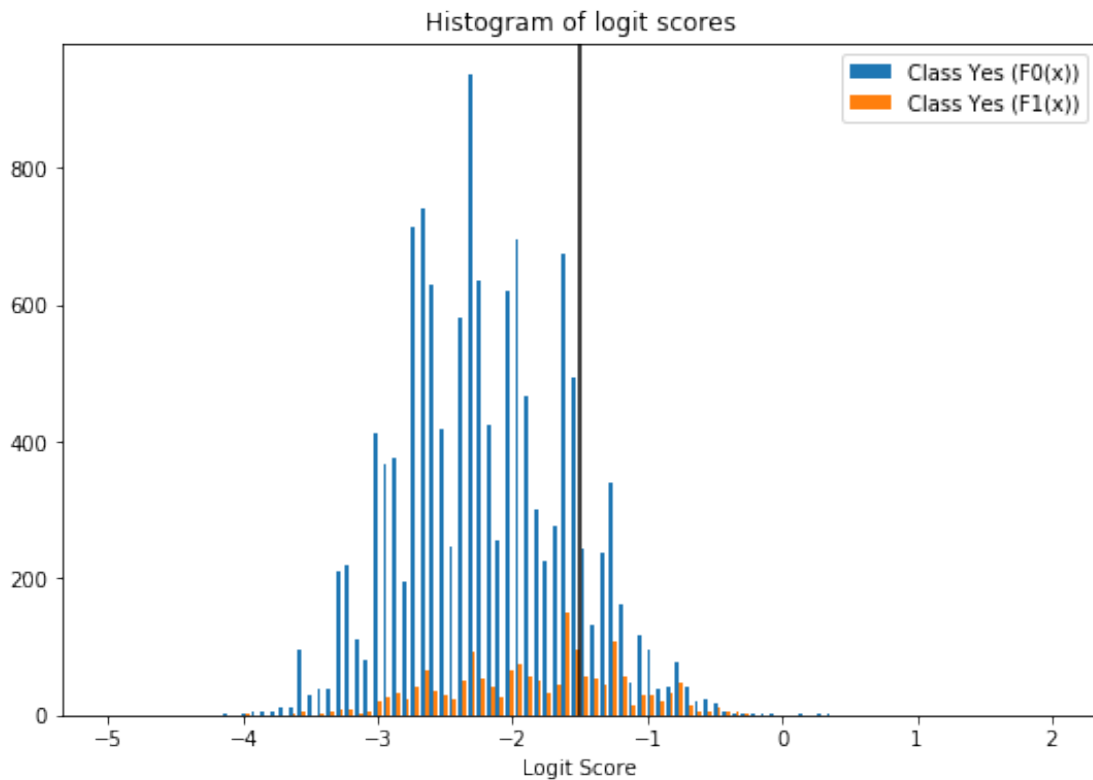
(c)

The left part of the black line is classified as YES, and the right part is classified as NO. True Postive Rate (TPR): percentage of the orange bars to the right of threshold. Flase Postive Rate (FPR): percentage of the blue bars to the right of threshold. By this definition, we have:

$$TPR = \int_T^{\infty} f_1(x)dx \quad (1)$$

$$FPR = \int_T^{\infty} f_0(x)dx \quad (2)$$

```
In [8]: plt.hist([y_0, y_1], 100, label=['Class Yes (F0(x))', 'Class Yes (F1(x))'],range=[-5,2],
plt.axvline(x=-1.5,color='black')
plt.xlabel('Logit Score')
plt.title(r'Histogram of logit scores')
plt.legend()
plt.show()
```



(d)

Let $FPR = f(t)$, $TPR = g(t)$ Indert into the formula given by the hint, we get: $AUC = \int_{-\infty}^{\infty} TPR(t)FPR'(t)dt$

(e)

Since, $TPR = \int_T^{\infty} f_1(x)dx$, $FPR = \int_T^{\infty} f_0(x)dx$

$$AUC = \int_{-\infty}^{\infty} f_1(t)f_0'(t)dt \quad (3)$$

$$= \int_{-\infty}^{\infty} \int_T^{\infty} f_1(s_1)f_0(s_0)ds_1ds_0 \quad (4)$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} (I_{S_1 > S_0})f_1(s_1)f_0(s_0)ds_1ds_0 \quad (5)$$

$$= \mathbb{E}(I_{S_1 > S_0}) \quad (6)$$

$$= \mathbb{P}(S_1 > S_0) \quad (7)$$

2 Problem 2

(a)

```

In [47]: #You may find it useful to use boolean expressions like (truth)&(~estimate)
         #This function should take in numpy logical arrays for estimate and truth
         #and single values for the losses
         def eval_estimate(estimate, truth, loss_FP, loss_FN):
             #Things that you write go here
             TP = estimate & truth
             TN = (~estimate) & (~truth)
             FP = estimate & (~truth)
             FN = (~estimate) & truth
             T = np.sum(truth)
             F = np.sum(~truth)

             sens = np.sum(TP)/(np.sum(TP)+np.sum(FN))
             spec = np.sum(TN)/(np.sum(TN)+np.sum(FP))
             loss = loss_FP * np.sum(FP) + loss_FN * np.sum(FN)
             return(sens, spec, loss)
         eval_estimate(logit_score>-1, ytest, 5, 100)

```

```

Out[47]: (0.09883720930232558, 0.9731818181818181, 156770)

```

(b)

```

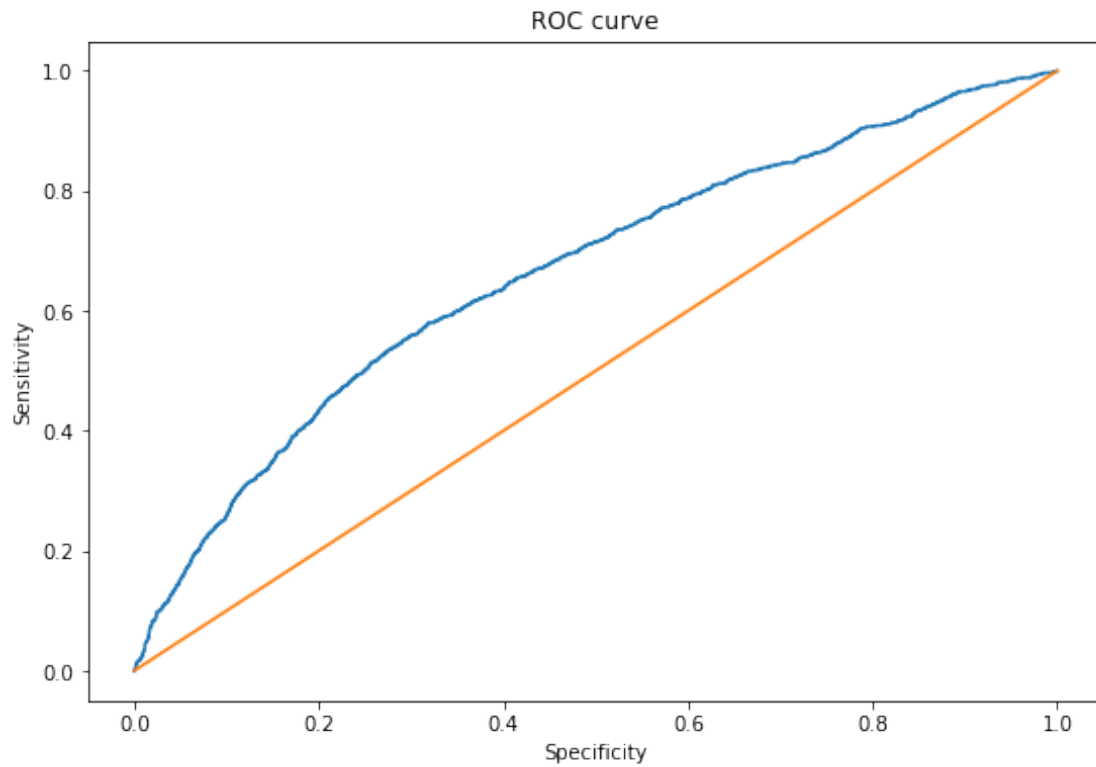
In [53]: # youll actually want to plot 1 specificity on the x-axis.
         # Add a diagonal line to compare to
         unique_values = np.sort(np.unique(logit_score))
         midpoints = (unique_values[0:(len(unique_values)-1)]+\
                       unique_values[1:len(unique_values)])/2.0
         sens, spec, loss = np.vectorize(lambda x: eval_estimate(logit_score>x, np.array(ytest
         plt.plot(1-spec, sens)
         plt.plot(sens,sens)
         plt.title('ROC curve')
         plt.xlabel('Specificity')
         plt.ylabel('Sensitivity')

```

```

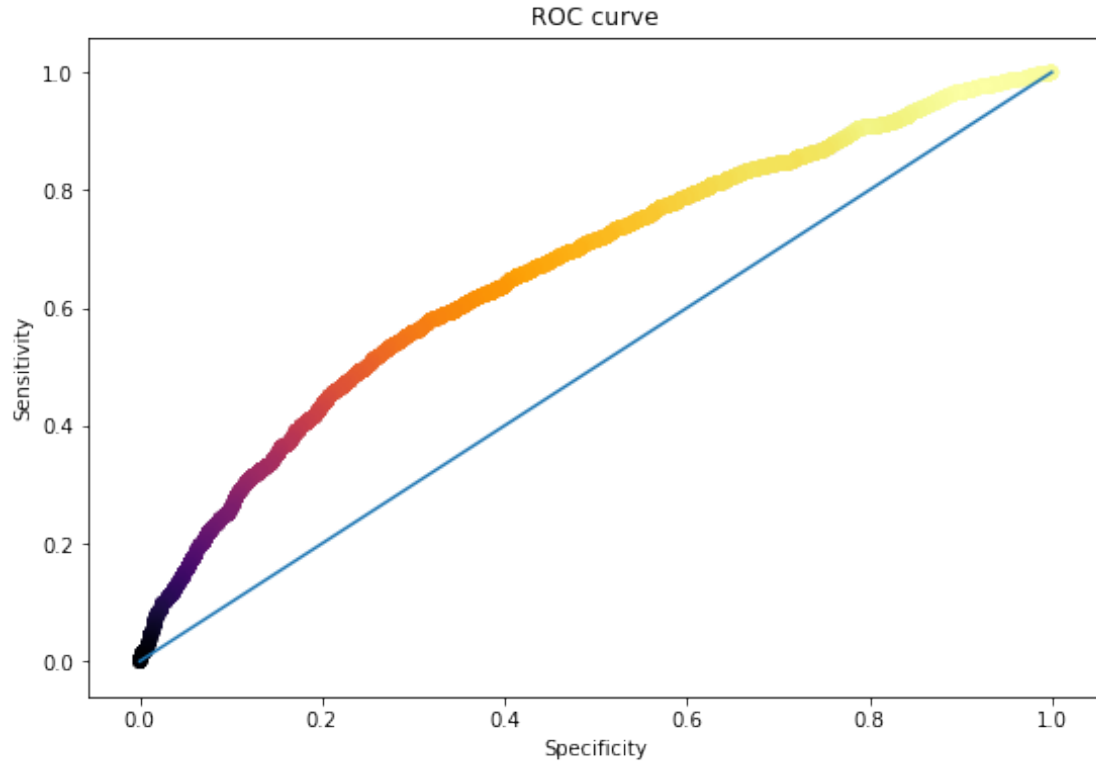
Out[53]: Text(0, 0.5, 'Sensitivity')

```



(c)

```
In [54]: from matplotlib import cm #For nice colors
plt.scatter(1-spec, sens, c= -loss, cmap=cm.inferno)
plt.plot(sens,sens)
plt.title('ROC curve')
plt.xlabel('Specificity')
plt.ylabel('Sensitivity')
plt.show()
```



(d) The line passing through the minimizing point with slope $\frac{N}{P} * \frac{L_{FP}}{L_{FN}}$ is tangent to the ROC curve.

```
In [55]: def abline(slope, intercept):
    axes = plt.gca()
    x_vals = np.array(axes.get_xlim())
    y_vals = intercept + slope*x_vals
    plt.plot(x_vals,y_vals,'--')

    min_idx = np.argmin(loss)
    min_sens = sens[min_idx]
    min_spec = spec[min_idx]

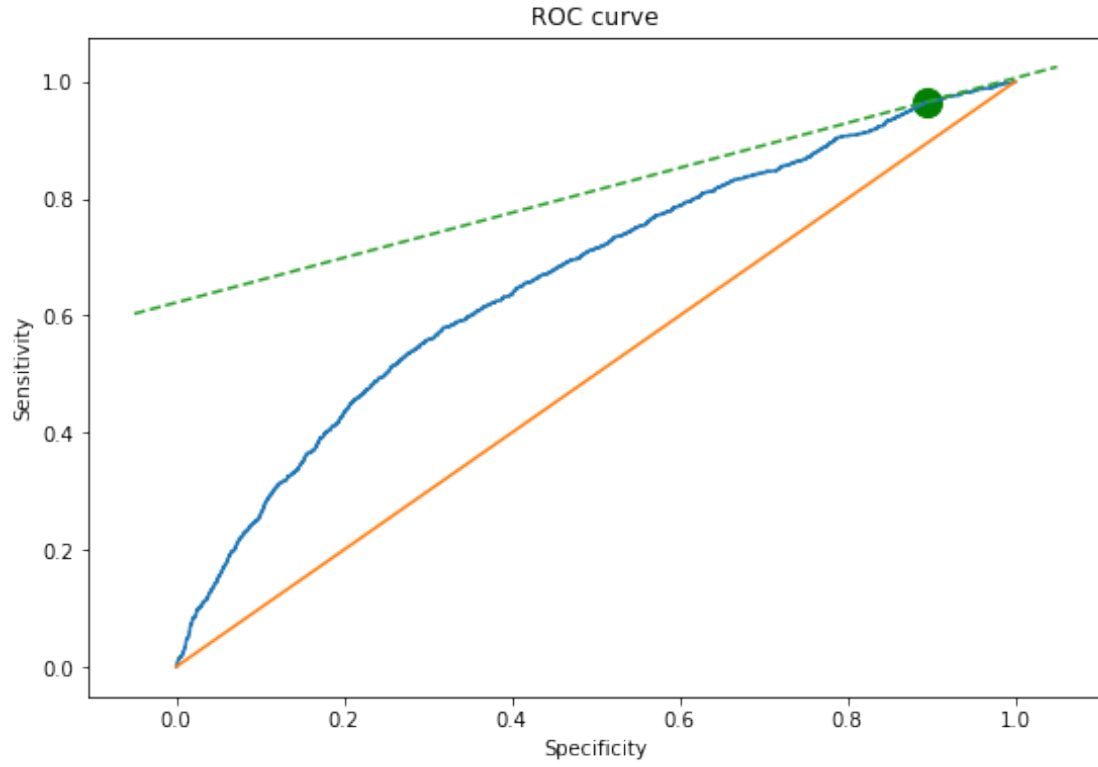
    plt.plot(1-spec,sens)
    plt.plot(sens,sens)
    plt.scatter(1-min_spec, min_sens, color='green', s=200)

    L_FN = 100
    L_FP = 5
    N = len(ytest)-np.sum(ytest)
    P = np.sum(ytest)
    slope = (N/P)*(L_FP/L_FN)
    intercept = min_sens-slope*(1-min_spec)
    abline(slope,intercept)
```



```
plt.title('ROC curve')
plt.xlabel('Specificity')
plt.ylabel('Sensitivity')
```

Out[55]: Text(0, 0.5, 'Sensitivity')



(e)

(i)

$$FPR = \frac{FP}{N} \quad (8)$$

$$FNR = \frac{FN}{P} \quad (9)$$

$$TotalLoss = FP * L_{FP} + FN * L_{FN} \quad (10)$$

$$= N * FPR * L_{FP} + P * FNR * L_{FN} \quad (11)$$

(ii)

$$FPR = \int_T^\infty f_0(x)dx \quad (12)$$

$$FNR = \int_{-\infty}^T f_1(x)dx \quad (13)$$

$$TotalLoss = \int_T^\infty NL_{FP}f_0(x)dx + \int_{-\infty}^T PL_{FN}f_1(x)dx \quad (14)$$

(iii)

Set $\frac{\partial Loss}{\partial T} = 0$

$$LHS = NL_{FP}(-f_0(T)) + PL_{FN}f_1(T) = 0 \quad (15)$$

$$\frac{f_1(T)}{f_0(T)} = \frac{NL_{FP}}{PL_{FN}} \quad (16)$$

$$= \frac{N}{P} \frac{L_{FP}}{L_{FN}} \quad (17)$$

, which is the same as that given by 2(d)

3 Problem 3

(a)

```
In [61]: optimal_threshold = L_FP/(L_FP + L_FN)
         optimal_logit_threshold = np.log(optimal_threshold/(1-optimal_threshold))
         min_sens2, min_specs2 ,min_loss2 = eval_estimate(logit_score>optimal_logit_threshold,
         #sens_o, spec_o, loss_o = eval_estimate(logit_score>-1, ytest, 5, 100)
         print('Loss at optimal threshold:',min_loss2)
         print('Minimum loss found in problem 2c:',loss[min_idx])
```

Loss at optimal threshold: 65245

Minimum loss found in problem 2c: 64950

(b) The new point, which is marked in red, has higher spec and sens compared to the true minimum point, which is marked in green. When we decide the optimal threshold for the red dot, we are minimizing the expected loss, but we are minimizing true loss for when we diciding threshold for the green dot.

```
In [67]: plt.plot(1 - spec, sens)
         plt.plot(sens,sens)
         plt.scatter(1-min_spec, min_sens, color='green', s=150)
         plt.scatter(1-min_specs2, min_sens2, color='red', s=150)

         P = np.sum(ytest)
         N = len(ytest)-np.sum(ytest)
```

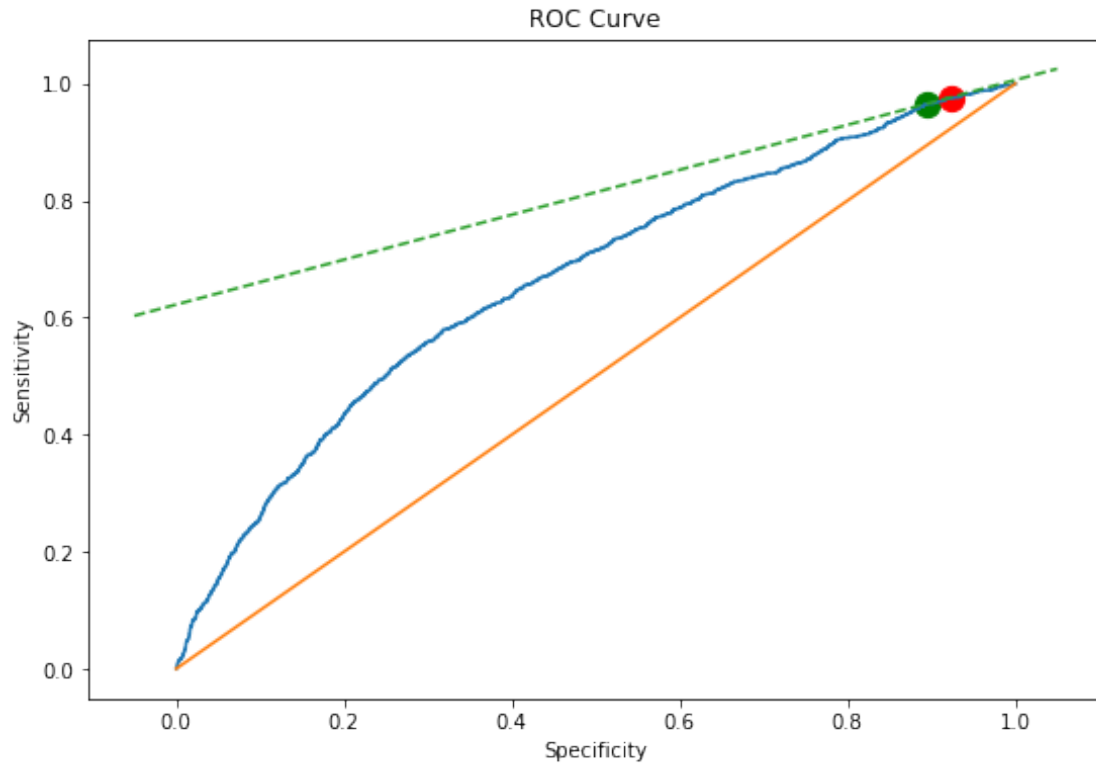
```

slope = (N/P)*(5.0/100)
intercept = min_sens-slope*(1-min_spec)
abline(slope,intercept)

plt.xlabel('Specificity')
plt.ylabel('Sensitivity')
plt.title('ROC Curve')

```

Out[67]: Text(0.5, 1.0, 'ROC Curve')



(c) As we can see from the graphs below, the classifier is well calibrated when predicted probability is below 0.3, but poorly calibrated as predicted probability increases. That's because the training data is unbalanced. So, for the threshold 0.5, we don't have enough observed frequencies for that part.

```

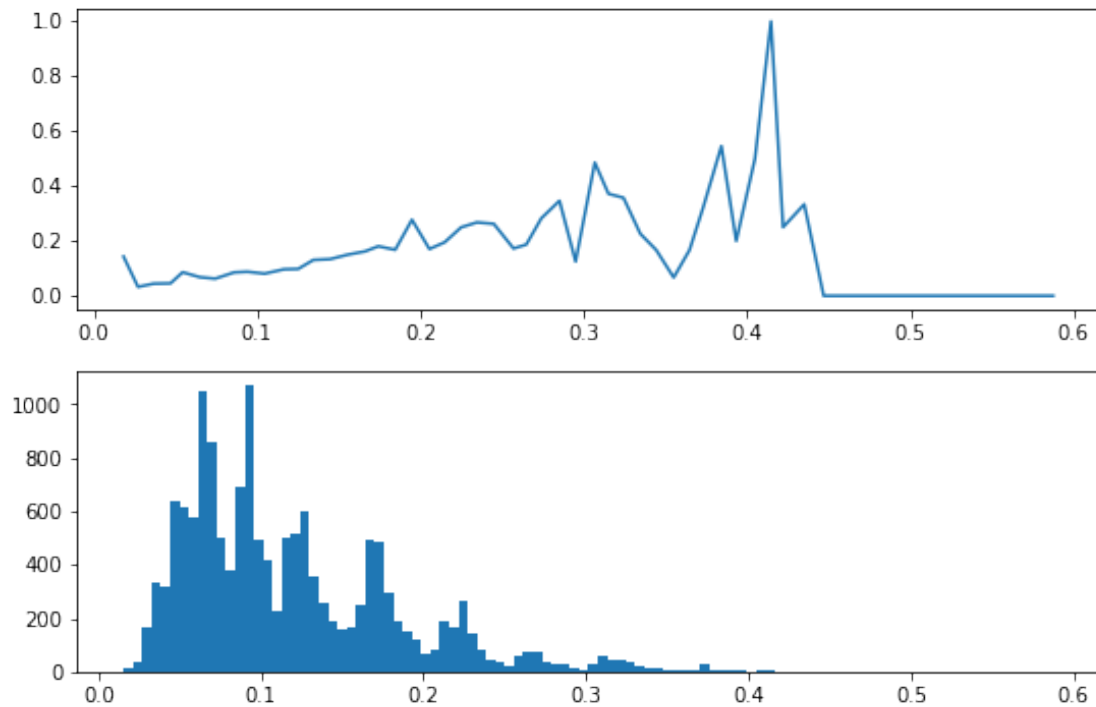
In [70]: # You can use the calibration curve function from sklearn.calibration and then plot t
# plot a histogram of the probabilities from your logistic regression
# on a second subfigure below your calibration plot
prob_true, prob_pred = calibration_curve(ytest, y_prob, n_bins = 100)
plt.subplot(211)
plt.plot(prob_pred, prob_true)
plt.subplot(212)
plt.hist(y_prob, bins = 100)

```

```

Out[70]: (array([1.200e+01, 3.600e+01, 1.660e+02, 3.320e+02, 3.180e+02, 6.410e+02,
        6.150e+02, 5.800e+02, 1.046e+03, 8.570e+02, 5.000e+02, 3.790e+02,
        6.940e+02, 1.075e+03, 4.940e+02, 4.150e+02, 2.280e+02, 5.020e+02,
        5.190e+02, 5.980e+02, 3.600e+02, 2.540e+02, 1.920e+02, 1.620e+02,
        1.680e+02, 2.520e+02, 4.960e+02, 4.890e+02, 2.990e+02, 1.890e+02,
        1.490e+02, 1.200e+02, 7.000e+01, 8.400e+01, 1.920e+02, 1.640e+02,
        2.630e+02, 1.420e+02, 8.500e+01, 4.700e+01, 3.300e+01, 1.900e+01,
        5.900e+01, 7.700e+01, 7.500e+01, 4.000e+01, 3.100e+01, 2.800e+01,
        1.000e+01, 8.000e+00, 3.200e+01, 5.800e+01, 4.600e+01, 4.300e+01,
        3.500e+01, 1.900e+01, 1.500e+01, 1.200e+01, 9.000e+00, 8.000e+00,
        4.000e+00, 3.000e+00, 2.900e+01, 7.000e+00, 6.000e+00, 5.000e+00,
        3.000e+00, 1.000e+00, 3.000e+00, 4.000e+00, 2.000e+00, 2.000e+00,
        1.000e+00, 2.000e+00, 0.000e+00, 1.000e+00, 0.000e+00, 2.000e+00,
        0.000e+00, 0.000e+00, 0.000e+00, 1.000e+00, 0.000e+00, 0.000e+00,
        0.000e+00, 0.000e+00, 0.000e+00, 0.000e+00, 0.000e+00, 0.000e+00,
        1.000e+00, 0.000e+00, 0.000e+00, 0.000e+00, 0.000e+00, 1.000e+00,
        0.000e+00, 0.000e+00, 0.000e+00, 1.000e+00]),
array([0.01553591, 0.02125602, 0.02697612, 0.03269623, 0.03841634,
        0.04413645, 0.04985655, 0.05557666, 0.06129677, 0.06701688,
        0.07273698, 0.07845709, 0.0841772 , 0.08989731, 0.09561742,
        0.10133752, 0.10705763, 0.11277774, 0.11849785, 0.12421795,
        0.12993806, 0.13565817, 0.14137828, 0.14709838, 0.15281849,
        0.1585386 , 0.16425871, 0.16997881, 0.17569892, 0.18141903,
        0.18713914, 0.19285924, 0.19857935, 0.20429946, 0.21001957,
        0.21573967, 0.22145978, 0.22717989, 0.2329 , 0.2386201 ,
        0.24434021, 0.25006032, 0.25578043, 0.26150053, 0.26722064,
        0.27294075, 0.27866086, 0.28438096, 0.29010107, 0.29582118,
        0.30154129, 0.30726139, 0.3129815 , 0.31870161, 0.32442172,
        0.33014182, 0.33586193, 0.34158204, 0.34730215, 0.35302226,
        0.35874236, 0.36446247, 0.37018258, 0.37590269, 0.38162279,
        0.3873429 , 0.39306301, 0.39878312, 0.40450322, 0.41022333,
        0.41594344, 0.42166355, 0.42738365, 0.43310376, 0.43882387,
        0.44454398, 0.45026408, 0.45598419, 0.4617043 , 0.46742441,
        0.47314451, 0.47886462, 0.48458473, 0.49030484, 0.49602494,
        0.50174505, 0.50746516, 0.51318527, 0.51890537, 0.52462548,
        0.53034559, 0.5360657 , 0.5417858 , 0.54750591, 0.55322602,
        0.55894613, 0.56466623, 0.57038634, 0.57610645, 0.58182656,
        0.58754666])),
<a list of 100 Patch objects>)

```



(d)

```
In [76]: logam = LogisticGAM().gridsearch(xtrain.values, ytrain, lam = np.logspace(-5, 3, 30),
```

```
100% (30 of 30) |#####| Elapsed Time: 0:08:52 Time: 0:08:52
```

```
In [108]: log_y_pred = logam.predict(xtest)
logypred_dens = logam.predict_proba(xtest)
logit_score_gam = np.log(logypred_dens / (1 - logypred_dens))
gam_indicies = np.sort(np.unique(logit_score_gam))
midpoints = (gam_indicies[0:(len(gam_indicies) - 1)] + gam_indicies[1:len(gam_indicies)])
sens_gam, spec_gam, loss_gam = np.vectorize(lambda x: eval_estimate(logit_score_gam, x))(midpoints)
#logistic minimum loss:
min_spec_log = spec[np.argmin(loss)]
min_sens_log = sens[np.argmin(loss)]
#gam min loss:
min_spec_gam = spec_gam[np.argmin(loss_gam)]
min_sens_gam = sens_gam[np.argmin(loss_gam)]

plt.figure()
x = np.random.rand(1000)
y = x
plt.plot(1-spec_gam, sens_gam, label = 'GAM ROC')
```

```

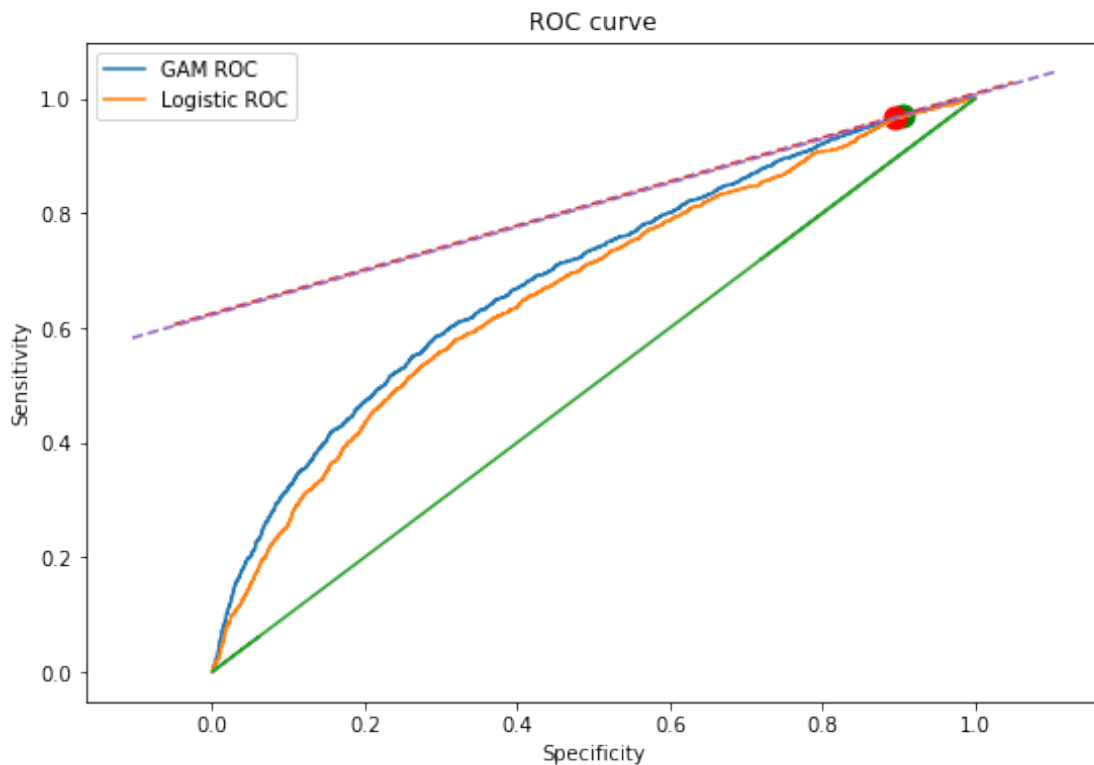
plt.plot(1-spec, sens, label = 'Logistic ROC')
plt.legend()
plt.xlabel('Specificity')
plt.ylabel('Sensitivity')
plt.title('ROC curve')
plt.plot(x,y,label = 'y=x')

plt.scatter(1-min_spec_gam, min_sens_gam, color='green', s=120)
intercept_gam = min_sens_gam-slope*(1-min_spec_gam)
abline(slope,intercept_gam)

plt.scatter(1-min_spec_log, min_sens_log, color='red', s=120)
intercept = min_sens_log-slope*(1-min_spec_log)
slope = L_FP/L_FN * (len(ytest)-np.sum(ytest))/np.sum(ytest)
abline(slope,intercept)

plt.show()

```



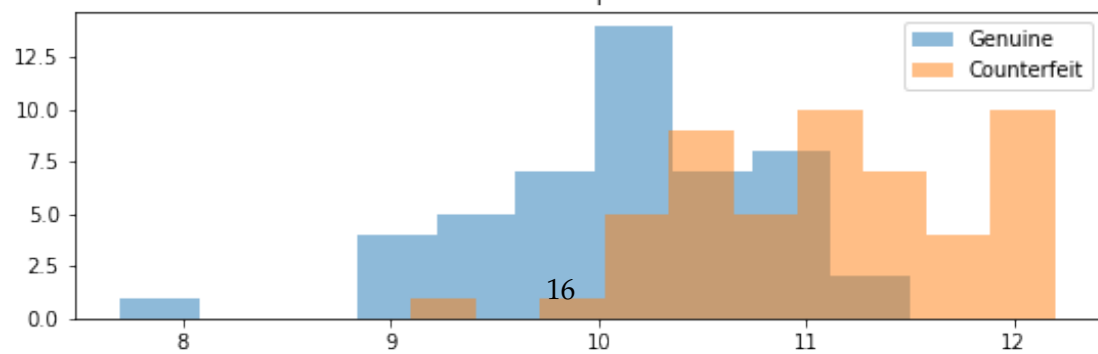
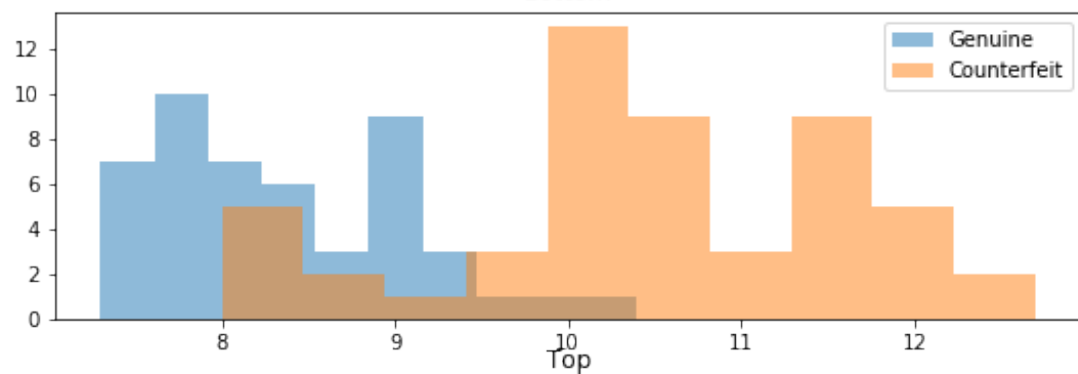
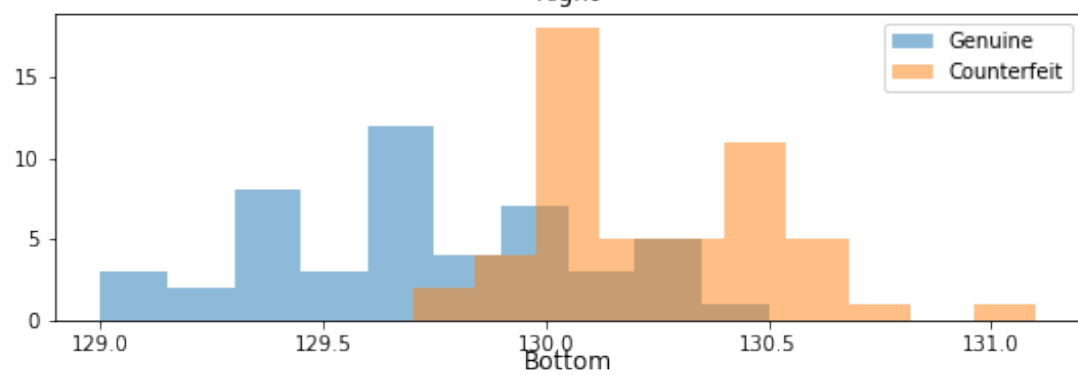
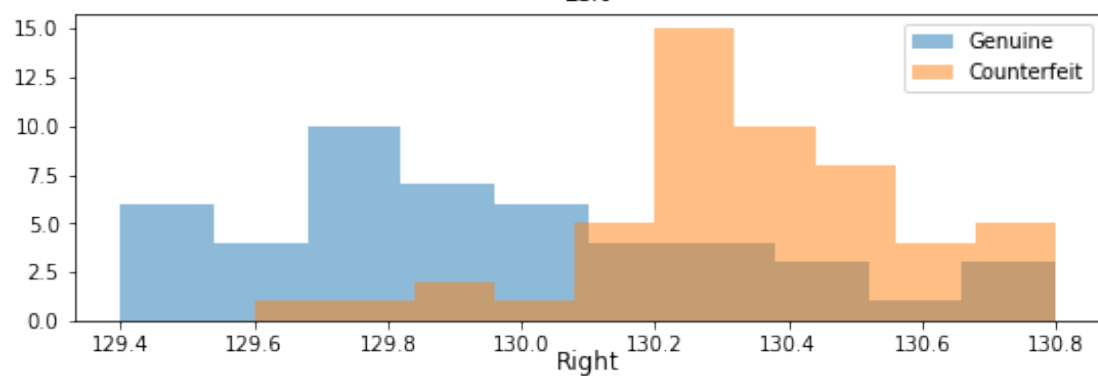
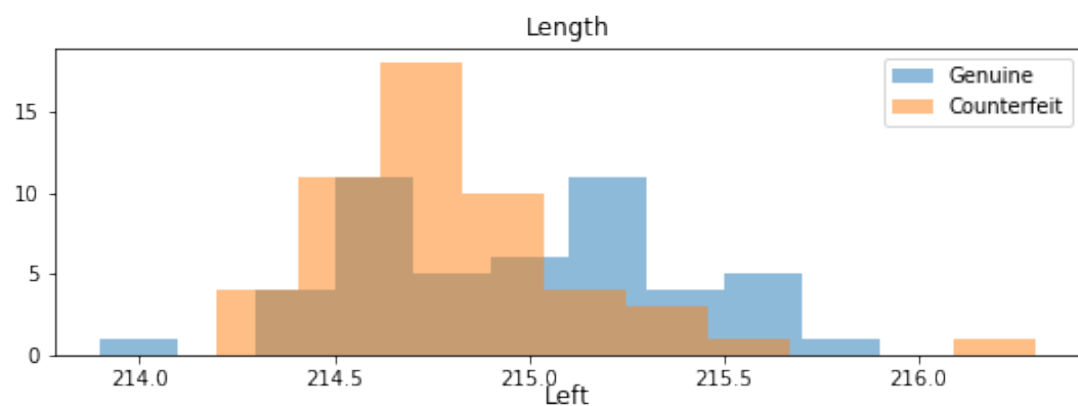
4 Problem 4

(a) We notice that none of the features are perfectly separated by the two classes.

```
In [84]: banknote = pd.read_csv('banknote_measurements.csv')
        X = banknote.iloc[:,2:]
        X = pd.get_dummies(X)
        y = np.where(banknote['counterfeit']=='genuine',0,1)
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=42)

In [99]: matplotlib.rcParams['figure.figsize'] = (9,16)

In [100]: fig, axs = plt.subplots(5,1)
          titles = ['Length', 'Left', 'Right', 'Bottom', 'Top']
          for i, ax in enumerate(axs):
              ax.hist(X_test[titles[i]][y_test==0], alpha=0.5, label='Genuine')
              ax.hist(X_test[titles[i]][y_test==1], alpha=0.5, label='Counterfeit')
              ax.legend()
              ax.set_title(titles[i])
```



(b)

```
In [95]: LDA = LinearDiscriminantAnalysis(n_components = 2)
fit = LDA.fit(X_train,y_train)
y_pred = fit.predict(X_test)
confusion_mrx = confusion_matrix(y_test,y_pred)
print(confusion_mrx)
print('Misclassification rate is: ', np.sum(y_test != y_pred)/len(y_test))
```

```
[[47  1]
 [ 6 46]]
```

Misclassification rate: 0.07

(c) As we can see from the graph below, the transformed space almost separate the classes perfectly. Only minor misclassification occurred.

```
In [97]: matplotlib.rcParams['figure.figsize'] = (9,6)
```

```
In [98]: X_trans = fit.transform(X_test)
plt.hist(X_trans[y_test==0], alpha=0.5, label='Genuine')
plt.hist(X_trans[y_test==1], alpha=0.5, label='Counterfeit')
plt.legend()
```

```
Out[98]: <matplotlib.legend.Legend at 0x1a265bb080>
```

